

Chapter 9

Security

9.1 Chapter Objectives

1. Explain the client-side security model for enterprise applications
2. Given an architectural system specification, select appropriate locations for implementation of specified security features, and select suitable technologies for implementation of those features.
3. Identify and classify potential threats to a system and describe how a given architecture will address the threats.
4. Describe and apply the commonly used declarative and programmatic methods used to secure applications.

9.2 Introduction

This chapter introduces basic security concepts and security implementation mechanisms.

9.2.1 Security Functions

A properly implemented security mechanism will provide the following functionality:

1. Prevent unauthorized access to application functions and business or personal data (authentication)
2. Hold system users accountable for operations they perform (non-repudiation)
3. Protect a system from service interruptions and other breaches that affect quality of service (QoS)

Ideally, properly implemented security mechanisms will also provide the following functionality:

1. Easy to administer
2. Transparent to system users
3. Interoperable across application and enterprise boundaries

9.2.2 Characteristics of Application Security

Java EE applications consist of components that can contain both protected and unprotected resources. Often, you need to protect resources to ensure that only authorized users have access. Authorization provides controlled access to protected resources. Authorization is based on identification and authentication. Identification is a process that enables recognition of an entity by a system, and authentication is a process that verifies the identity of a user, device, or other entity in a computer system, usually as a prerequisite to allowing access to resources in a system.

Authorization and authentication are not required for an entity to access unprotected resources. Accessing a resource without authentication is referred to as unauthenticated or anonymous access.

The characteristics of application security that, when properly addressed, help to minimize the security threats faced by an enterprise, include the following:

1. **Authentication:** The means by which communicating entities (for example, client and server) prove to one another that they are acting on behalf of specific identities that are authorized for access. This ensures that users are who they say they are.
2. **Authorization, or Access Control:** The means by which interactions with resources are limited to collections of users or programs for the purpose of enforcing integrity, confidentiality, or availability constraints. This ensures that users have permission to perform operations or access data.
3. **Data integrity:** The means used to prove that information has not been modified by a third party (some entity other than the source of the information). For example, a recipient of data sent over an open network must be able to detect and discard messages that were modified after they were sent. This ensures that only authorized users can modify data.
4. **Confidentiality or Data Privacy:** The means used to ensure that information is made available only to users who are authorized to access it. This ensures that only authorized users can view sensitive data.
5. **Non-repudiation:** The means used to prove that a user performed some action such that the user cannot reasonably deny having done so. This ensures that transactions can be proven to have happened.
6. **Quality of Service (QoS):** The means used to provide better service to selected network traffic over various technologies.
7. **Auditing:** The means used to capture a tamper-resistant record of security-related events for the purpose of being able to evaluate the effectiveness of security policies and mechanisms. To enable this, the system maintains a record of transactions and security information.

9.3 Working with Realms, Users, Groups, and Roles

You often need to protect resources to ensure that only authorized users have access. Authorization provides controlled access to protected resources. Authorization is based on identification and authentication. Identification is a process that enables recognition of an entity by a system, and authentication is a process that verifies the identity of a user, device, or other entity in a computer system, usually as a prerequisite to allowing access to resources in a system. These concepts are discussed in more detail in Characteristics of Application Security.

This section discusses setting up users so that they can be correctly identified and either given access to protected resources, or denied access if the user is not authorized to access the protected resources. To authenticate a user, you need to follow these basic steps:

1. The Application Developer writes code to prompt the user for their user name and password.
2. The Application Developer communicates how to set up security for the deployed application by use of a deployment descriptor or metadata annotation.
3. The Server Administrator sets up authorized users and groups on the Enterprise Server.
4. The Application Deployer maps the applications security roles to users, groups, and principals defined on the Enterprise Server.

9.3.1 What Are Realms, Users, Groups, and Roles?

A realm is a security policy domain defined for a web or application server. It is also a string, passed as part of an HTTP request during basic authentication, that defines a protection space. The protected resources on a server can be partitioned into a set of protection spaces, each with its own authentication scheme and/or authorization database containing a collection of users, which may or may not be assigned to a group.

An application will often prompt a user for their user name and password before allowing access to a protected resource. After the user has entered their user name and password, that information is passed to the server, which either authenticates the user and sends the protected resource, or does not authenticate the user, in which case access to the protected resource is denied.

In some applications, authorized users are assigned to roles. In this situation, the role assigned to the user in the application must be mapped to a principal or group defined on the application server.

The following sections provide more information on realms, users, groups, and roles.

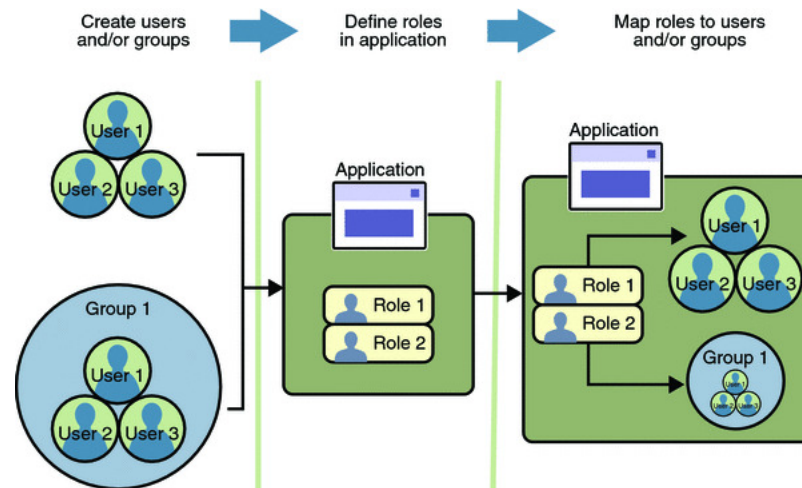


FIGURE 9.1: Security Role Mapping

9.3.1.1 What Is a Realm?

A realm is a security policy domain defined for a web or application server. It is also a string, passed as part of an HTTP request during basic authentication, that defines a protection space. The protected resources on a server can be partitioned into a set of protection spaces, each with its own authentication scheme and/or authorization database containing a collection of users, which may or may not be assigned to a group.

For a web application, a realm is a complete database of users and groups that identify valid users of a web application (or a set of web applications) and are controlled by the same authentication policy.

The authentication service can govern users in multiple realms.

9.3.1.2 What Is a User?

A user is an individual (or application program) identity that has been defined in the Enterprise Server. In a web application, a user can have a set of roles associated with that identity, which entitles them to access all resources protected by those roles. Users can be associated with a group.

9.3.1.3 What Is a Group?

A group is a set of authenticated users, classified by common traits, defined in the Enterprise Server.

9.3.1.4 What Is a Role?

A role is an abstract name for the permission to access a particular set of resources in an application. A role can be compared to a key that can open a lock. Many people might have a copy of the key. The lock doesn't care who you are, only that you have the right key.

9.3.1.5 Some Other Terminology

The following terminology is also used to describe the security requirements of the Enterprise Applications:

-
- **Principal:** A principal is an entity that can be authenticated by an authentication protocol in a security service that is deployed in an enterprise. A principal is identified using a principal name and authenticated using authentication data.
- **Security policy domain (also known as security domain or realm):** A security policy domain is a scope over which a common security policy is defined and enforced by the security administrator of the security service.
- **Security attributes:** A set of security attributes is associated with every principal. The security attributes have many uses, for example, access to protected resources and auditing of users. Security attributes can be associated with a principal by an authentication protocol.
- **Credential:** A credential contains or references information (security attributes) used to authenticate a principal for the product services. A principal acquires a credential upon authentication, or from another principal that allows its credential to be used.

9.4 Establishing a Secure Connection Using SSL

Secure Socket Layer (SSL) technology is security that is implemented at the transport layer. SSL allows web browsers and web servers to communicate over a secure connection. In this secure connection, the data that is being sent is encrypted before being sent and then is decrypted upon receipt and before processing. Both the browser and the server encrypt all traffic before sending any data. SSL addresses the following important security considerations.

1. **Authentication:** During your initial attempt to communicate with a web server over a secure connection, that server will present your web browser with a set of credentials in the form of a server certificate. The purpose of the certificate is to verify that the site is who and what it claims to be. In some cases, the server may request a certificate that the client is who and what it claims to be (which is known as client authentication).
2. **Confidentiality:** When data is being passed between the client and the server on a network, third parties can view and intercept this data. SSL responses are encrypted so that the data cannot be deciphered by the third party and the data remains confidential.
3. **Integrity:** When data is being passed between the client and the server on a network, third parties can view and intercept this data. SSL helps guarantee that the data will not be modified in transit by that third party.

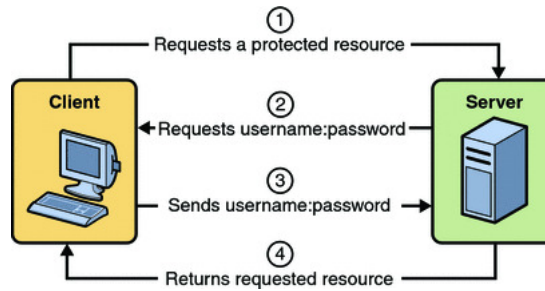


FIGURE 9.2: Basic Security Authentication Mechanism

9.5 Authentication Mechanisms

The choices for authentication mechanisms are available to help one secure web applications:

1. HTTP Basic Authentication
2. Form-Based Authentication
3. HTTPS Client Authentication
4. Digest Authentication

9.5.1 HTTP Basic Authentication

Specifying HTTP Basic Authentication requires that the server request a user name and password from the web client and verify that the user name and password are valid by comparing them against a database of authorized users in the specified or default realm.

When basic authentication is declared, the following actions occur:

1. A client requests access to a protected resource.
2. The web server returns a dialog box that requests the user name and password.
3. The client submits the user name and password to the server.
4. The server authenticates the user in the specified realm and, if successful, returns the requested resource.

HTTP basic authentication is not a secure authentication mechanism. Basic authentication sends user names and passwords over the Internet as text that is Base64 encoded, and the target server is not authenticated. This form of authentication can expose user names and passwords. If someone can intercept the transmission, the user name and password information can easily be decoded. However, when a secure transport mechanism, such as SSL, or security at the network level, such as the IPSEC protocol or VPN strategies, is used in conjunction with basic authentication, some of these concerns can be alleviated.

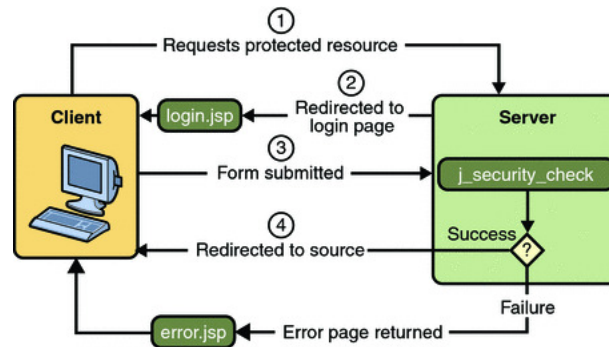


FIGURE 9.3: Security Form Based Authentication

9.5.2 Form-Based Authentication

Form-based authentication allows the developer to control the look and feel of the login authentication screens by customizing the login screen and error pages that an HTTP browser presents to the end user. When form-based authentication is declared, the following actions occur:

1. A client requests access to a protected resource.
2. If the client is unauthenticated, the server redirects the client to a login page.
3. The client submits the login form to the server.
4. The server attempts to authenticate the user.
 - (a) If authentication succeeds, the authenticated users principal is checked to ensure it is in a role that is authorized to access the resource. If the user is authorized, the server redirects the client to the resource using the stored URL path.
 - (b) If authentication fails, the client is forwarded or redirected to an error page.

Form-based authentication is not particularly secure. In form-based authentication, the content of the user dialog box is sent as plain text, and the target server is not authenticated. This form of authentication can expose your user names and passwords unless all connections are over SSL. If someone can intercept the transmission, the user name and password information can easily be decoded. However, when a secure transport mechanism, such as SSL, or security at the network level, such as the IPSEC protocol or VPN strategies, is used in conjunction with form-based authentication, some of these concerns can be alleviated. To add a protected transport in your application, use the elements described in Specifying a Secure Connection.

9.5.2.1 Using Login Forms

When creating a form-based login, be sure to maintain sessions using cookies or SSL session information.

As shown in Form-Based Authentication, for authentication to proceed appropriately, the action of the login form must always be **j_security_check**. This restriction is made so that the login form will work no matter which resource it is for, and to avoid requiring the server to specify the action field of the outbound form. The following code snippet shows how the form should be coded into the HTML page:

```
<form method="POST" action="j_security_check">  
<input type="text" name="j_username">  
<input type="password" name="j_password">  
</form>
```

9.5.3 HTTPS Client Authentication

HTTPS Client Authentication requires the client to possess a Public Key Certificate (PKC). If you specify client authentication, the web server will authenticate the client using the clients public key certificate.

HTTPS Client Authentication is a more secure method of authentication than either basic or form-based authentication. It uses HTTP over SSL (HTTPS), in which the server authenticates the client using the clients Public Key Certificate (PKC). Secure Sockets Layer (SSL) technology provides data encryption, server authentication, message integrity, and optional client authentication for a TCP/IP connection. You can think of a public key certificate as the digital equivalent of a passport. It is issued by a trusted organization, which is called a certificate authority (CA), and provides identification for the bearer.

9.5.3.1 Mutual Authentication

With mutual authentication, the server and the client authenticate one another. There are two types of mutual authentication:

1. Certificate-based mutual authentication
2. User name- and password-based mutual authentication

9.5.3.2 Certificate-based mutual authentication

When using certificate-based mutual authentication, the following actions occur:

1. A client requests access to a protected resource.
2. The web server presents its certificate to the client.
3. The client verifies the servers certificate.
4. If successful, the client sends its certificate to the server.

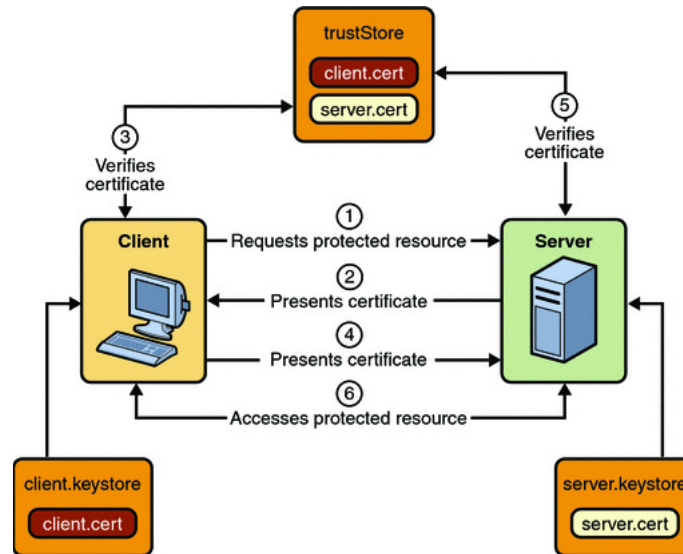


FIGURE 9.4: Certificate-Based Mutual Authentication

5. The server verifies the clients credentials.
6. If successful, the server grants access to the protected resource requested by the client.

9.5.3.3 User name- and password-based mutual authentication

In user name- and password-based mutual authentication, the following actions occur:

1. A client requests access to a protected resource.
2. The web server presents its certificate to the client.
3. The client verifies the servers certificate.
4. If successful, the client sends its user name and password to the server, which verifies the clients credentials.
5. If the verification is successful, the server grants access to the protected resource requested by the client.

9.5.4 Digest Authentication

Like HTTP Basic Authentication, HTTP Digest Authentication authenticates a user based on a username and a password. However, unlike HTTP Basic Authentication, HTTP Digest Authentication does not send user passwords over the network. In HTTP Digest authentication, the client sends a one-way cryptographic hash of the password (and additional data). Although passwords are not sent on the wire, HTTP Digest authentication requires that clear text password equivalents be available to the authenticating container so that it can validate received authenticators by calculating the expected digest.

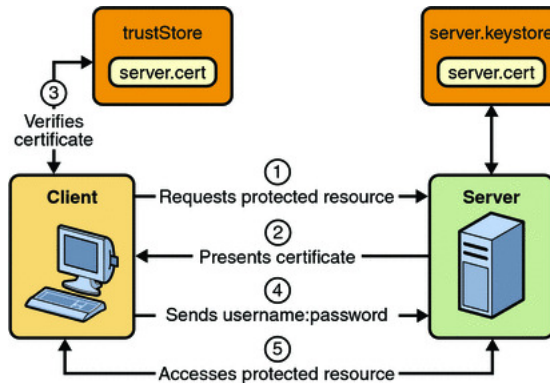


FIGURE 9.5: User Name- and Password-Based Mutual Authentication

9.6 Security With SpringFramework

The application we are using in the lab uses SpringFramework. To configure security you first need to setup two entities one with username and password and the other with ROLES that a user will be assigned once logged in. Below are excerpts of the two entities. In this class we are going to use a FORM BASED authentication mechanism

For the user:

```
public class Users implements Serializable {

    private static final long serialVersionUID = 1L;
    @Id
    @GeneratedValue(strategy = GenerationType.AUTO)
    private Long id;
    @Column(unique = true)
    private String username;
    private String passwd;
    private boolean enabled;
    @OneToMany(cascade = CascadeType.ALL)
    @JoinColumn(name = "role_id")
    private List<Roles> roles;
```

For the Roles:

```
public class Roles implements Serializable {

    private static final long serialVersionUID = 1L;
    @Id
    @GeneratedValue(strategy = GenerationType.AUTO)
    private Long id;
    private String roleName;
    private String username;
    @ManyToOne(cascade = CascadeType.ALL)
    private Users user;
```

Please note the relationship between the entities

Next we need to modify our application to add the Spring Security Filter as shown below in the web.xml

```

.....
<servlet-mapping>
    <servlet-name>CXFServlet</servlet-name>
    <url-pattern>/ws/*</url-pattern>
</servlet-mapping>

    <filter>
        <filter-name>springSecurityFilterChain</filter-name>

<filter-class>org.springframework.web.filter.DelegatingFilterProxy</filter-class>
    </filter>
    <filter-mapping>
        <filter-name>springSecurityFilterChain</filter-name>
        <url-pattern>/*</url-pattern>
    </filter-mapping>

    <servlet>
        <servlet-name>hashpayervlet</servlet-name>
        <servlet-class>
            com.vaadin.terminal.gwt.server.ApplicationServlet
        </servlet-class>
        <init-param>
            <param-name>application</param-name>
            <param-value>com.hashpay.clients.web.HashPayMain</param-value>
        </init-param>
    </servlet>
    <servlet-mapping>
        <servlet-name>hashpayervlet</servlet-name>
        <url-pattern>/app/*</url-pattern>
    </servlet-mapping>
    <servlet-mapping>
        <servlet-name>hashpayervlet</servlet-name>
        <url-pattern>/VAADIN/*</url-pattern>
    </servlet-mapping>
.....

```

Not also the the change has been made to the application context so that the user can not access it directly, but through a redirect call in the index file. Note that the app context has been added to the mapping

Next we create the index file with the redirect call as show below in the root folder of the web folder for the web resources.

```

<%@page contentType="text/html" pageEncoding="UTF-8"%>
<% response.sendRedirect("/university/app/"); %>

```

After the redirect, we create two files in the web resources folder, the login.jsp file, which will have the form and the accessDenied.jsp

```

<center>
  <form action="j_spring_security_check" method="POST">
    <label for="j_username">Username</label>
    <input type="text" name="j_username" id="j_username" />
    <br/>
    <label for="j_password">Password</label>
    <input type="password" name="j_password" id="j_password" />
    <br/>
    <input type='checkbox' name='_spring_security_remember_me' /> ←
    Remember me
    on this computer.
    <br/>
    <input type="submit" value="Login"/>
  </form>
</center>

```

accessDenied.jsp

```

<%@ taglib prefix="c" uri="http://java.sun.com/jsp/jstl/core"%>
<html>
<head>
  <meta http-equiv="Content-Type" content="text/html; charset=UTF-8" />
  <title>Foo</title>
</head>
<body>
  <p>You have tried to access a protected area of this application ←
  where
  you don't have rights.</p>
</body>
</html>

```

Lastly we create a the ApplicationContextsecurity.xml in the conf folder of our application. This is the file that forms the heart oof the security mechanism. A sample is shown below.

```

<!--SECTION A -->
<beans:beans xmlns="http://www.springframework.org/schema/security"
  xmlns:beans="http://www.springframework.org/schema/beans"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://www.springframework.org/schema/beans
    http://www.springframework.org/schema/beans/spring-beans-3.0.←
    xsd
    http://www.springframework.org/schema/security
    http://www.springframework.org/schema/security/spring-security-3.0.xsd">

<!-- SECTION B -->

  <global-method-security secured-annotations="enabled">

  </global-method-security>

<--! SECTITION C -->

  <http auto-config="true">

```

```

        <intercept-url pattern="/login.jsp*" access="IS.AUTHENTICATED.ANONYMOUSLY" />
        <intercept-url pattern="/VAADIN/*" filters="none" />
        <intercept-url pattern="/*.jsp" filters="none" />
        <intercept-url pattern="/app/*" access="ROLE.CUSTOMER" />
        <form-login login-page="/login.jsp" />
    </http>

<— SECTION D —>

    <authentication-manager alias="authenticationManager">
        <authentication-provider>
            <password-encoder hash="md5" />
            <jdbc-user-service data-source-ref="dataSource">
                users-by-username-query="SELECT username, passwd, enabled FROM
                Users u where u.username=?"
                authorities-by-username-query="SELECT u.username, r.rolename FROM
                Users u, Roles r
                WHERE u.id = r.role_id AND u.username=?" />
            </authentication-provider>
        </authentication-manager>
    </beans:beans>

```

The configuration file is divided into 4 Sections A, B, C, A

Section A deals with the namespace configuration. Section B is used to provide security annotations on the application classes. Section C is used to protect resources and set the form configurations and type of authentication mechanism. Section D define where the users and roles are stored in the database.

Lastly Make sure you have a Data Source in your ApplicationContext-connection.xml as shown below

```

<context:property-placeholder location="classpath:database.properties"/>
<bean id="dataSource" class="org.apache.commons.dbcp.BasicDataSource"
    destroy-method="close">
    <property name="driverClassName" value="${database.
    driverClassName}" />
    <property name="url" value="${database.url}" />
    <property name="username" value="${database.username}" />
    <property name="password" value="${database.password}" />
    <property name="maxActive" value="10" />
    <property name="maxIdle" value="3" />
    <property name="maxWait" value="5000" />
    <property name="defaultAutoCommit" value="true" />
</bean>

```

Notice that the database variables are stored in a properties file called database.properties. So you need to create this file in your root folder as shown below

```

# To change this template, choose Tools | Templates
# and open the template in the editor.
database.url=jdbc:derby://localhost:1527/sample
database.generateDdl=true
database.username=app
database.password=app
database.showSql=true

```

```
database.platform=org.hibernate.dialect.DerbyDialect
database.driverClassName=org.apache.derby.jdbc.ClientDriver
```

9.7 Spring Security Configurations

9.7.1 Security Namespace Configuration

A namespace element can be used simply to allow a more concise way of configuring an individual bean or, more powerfully, to define an alternative configuration syntax which more closely matches the problem domain and hides the underlying complexity from the user.

```
<beans:beans xmlns="http://www.springframework.org/schema/security"
  xmlns:beans="http://www.springframework.org/schema/beans"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://www.springframework.org/schema/beans
    http://www.springframework.org/schema/beans/spring-beans-3.0.xsd
    http://www.springframework.org/schema/security
    http://www.springframework.org/schema/security/spring-security-3.0.xsd">
  ...
</beans:beans>
```

9.7.2 Adding a Password Encoder

Often your password data will be encoded using a hashing algorithm. This is supported by the `password-encoder` element. With SHA encoded passwords, the original authentication provider configuration would look like this:

```
<authentication-manager>
  <authentication-provider>
    <password-encoder hash="sha"/>
    <user-service>
      <user name="jimi" password="d7e6351eaa13189a5a3641bab846c8e8c69ba39f"
        authorities="ROLE_USER, ROLE_ADMIN" />
      <user name="bob" password="e7421b1b8765d8f9406d87e7cc6aa784c4ab97f"
        authorities="ROLE_USER" />
    </user-service>
  </authentication-provider>
</authentication-manager>
```

Please Note that you have to save your password in the encoded format when you create a user. The code listing below shows you how to encode a password in MD5 in Java

```
import java.security.MessageDigest;
```

```

import java.security.NoSuchAlgorithmException;
import java.security.NoSuchProviderException;
import java.util.logging.Level;
import java.util.logging.Logger;

/**
 *
 * @author boniface
 */
public class PasswordEncrypt {

    public static String encrypt(String freeText) {
        StringBuffer getString = new StringBuffer();
        try {
            MessageDigest msg = MessageDigest.getInstance("MD5", "SUN");
            byte bs[] = freeText.getBytes();
            byte digest[] = msg.digest(bs);
            for (int i = 0; i < digest.length; ++i) {
                getString.append(Integer.toHexString(0x0100 + (digest[i] <
                & 0x00FF)).substring(1));
            }
        } catch (NoSuchAlgorithmException ex) {
            Logger.getLogger(PasswordEncrypt.class.getName()).log(Level.SEVERE, null, ex);
        } catch (NoSuchProviderException ex) {
            Logger.getLogger(PasswordEncrypt.class.getName()).log(Level.SEVERE, null, ex);
        }
        return getString.toString();
    }
}

```

and the you would set the password in your class as

```

....
Users user = new Users();
user.setPassword(new PasswordEncrypt.encrypt("test123"));
.....

```

9.7.3 Adding HTTP/HTTPS Channel Security

If your application supports both HTTP and HTTPS, and you require that particular URLs can only be accessed over HTTPS, then this is directly supported using the `requires-channel` attribute on

```

<intercept-url>:
  <http>
    <intercept-url pattern="/secure/**" access="ROLE_USER" requires-channel="https"/>
    <intercept-url pattern="/**" access="ROLE_USER" requires-channel="any"/>
    ...
  </http>

```

With this configuration in place, if a user attempts to access anything matching the "secure*" pattern using HTTP, they will first be redirected to an HTTPS URL. The available options are "http", "https" or "any". Using the value "any" means that either HTTP or HTTPS can be used. If your application uses non-standard ports for HTTP and/or HTTPS, you can specify a list of port mappings as follows:

```
<http>
...
<port-mappings>
  <port-mapping http="9080" https="9443" />
</port-mappings>
</http>
```

9.7.4 Concurrent Session Control

If you wish to place constraints on a single user's ability to log in to your application, Spring Security supports this out of the box with the following simple additions. First you need to add the following listener to your web.xml file to keep Spring Security updated about session lifecycle events:

```
<listener>
  <listener-class>
    org.springframework.security.web.session.HttpSessionEventPublisher
  </listener-class>
</listener>
```

Then add the following lines to your application context:

```
<http>
...
<session-management>
  <concurrency-control max-sessions="1" />
</session-management>
</http>
```

This will prevent a user from logging in multiple times - a second login will cause the first to be invalidated. Often you would prefer to prevent a second login, in which case you can use

```
<http>
...
<session-management>
  <concurrency-control max-sessions="1"
    error-if-maximum-exceeded="true" />
</session-management>
</http>
```

The second login will then be rejected. By rejected, we mean that the user will be sent to the authentication-failure-url if form-based login is being used. If the second authentication takes place through another non-interactive mechanism, such as remember-me,

an unauthorized (402) error will be sent to the client. If instead you want to use an error page, you can add the attribute `session-authentication-error-url` to the `session-management` element.

9.7.5 Session Fixation Attack Protection

Session fixation attacks are a potential risk where it is possible for a malicious attacker to create a session by accessing a site, then persuade another user to log in with the same session (by sending them a link containing the session identifier as a parameter, for example). Spring Security protects against this automatically by creating a new session when a user logs in. If you don't require this protection, or it conflicts with some other requirement, you

can control the behaviour using the `session-fixation-protection` attribute on `<session-management>`, which has three options

- `migrateSession` - creates a new session and copies the existing session attributes to the new session. This is the default.
- `none` - Don't do anything. The original session will be retained.
- `newSession` - Create a new "clean" session, without copying the existing session data.

9.7.6 Method Security

The `<global-method-security>` Element is used to enable annotation-based security in your application (by setting the appropriate attributes on the element), and also to group together security pointcut declarations which will be applied across your entire application context. You should only declare one `<global-method-security>` element. The following declaration would enable support for Spring Security's `@Secured`:

```
<global-method-security secured-annotations="enabled" />
```

Adding an annotation to a method (on an class or interface) would then limit the access to that method accordingly. Spring Security's native annotation support defines a set of attributes for the method. These will be passed to the `AccessDecisionManager` for it to make the actual decision:

```
public interface BankService {
    @Secured("IS_AUTHENTICATED_ANONYMOUSLY")
    public Account readAccount(Long id);
    @Secured("IS_AUTHENTICATED_ANONYMOUSLY")
    public Account[] findAccounts();
    @Secured("ROLE_TELLER")
    public Account post(Account account, double amount);
}
```

Support for JSR-250 annotations can be enabled using

```
<global-method-security jsr250-annotations="enabled" />
```

These are standards-based and allow simple role-based constraints to be applied but do not have the power Spring Security's native annotations. To use the new expression-based syntax, you would use

```
<global-method-security pre-post-annotations="enabled" />

and the equivalent Java code would be
public interface BankService {
    @PreAuthorize("isAnonymous()")
    public Account readAccount(Long id);
    @PreAuthorize("isAnonymous()")
    public Account[] findAccounts();
    @PreAuthorize("hasAuthority('ROLE_TELLER')")
    public Account post(Account account, double amount);
}
```

Expression-based annotations are a good choice if you need to define simple rules that go beyond checking the role names against the user's list of authorities. You can enable more than one type of annotation in the same application, but you should avoid mixing annotations types in the same interface or class to avoid confusion.

9.7.7 Obtaining information about the current user

Inside the `SecurityContextHolder` we store details of the principal currently interacting with the application. Spring Security uses an `Authentication` object to represent this information. You won't normally need to create an `Authentication` object yourself, but it is fairly common for users to query the `Authentication` object. You can use the following code block - from anywhere in your application - to obtain the name of the currently authenticated user, for example:

```
Object principal =
SecurityContextHolder.getContext().getAuthentication().getPrincipal();
if (principal instanceof UserDetails) {
    String username = ((UserDetails)principal).getUsername();
} else {
    String username = principal.toString();
}
```

The object returned by the call to `getContext()` is an instance of the `SecurityContext` interface. This is the object that is kept in thread-local storage. As we'll see below, most authentication mechanisms with Spring Security return an instance of `UserDetails` as the principal.

9.8 Chapter Exercise

Apply the Securty to your application with users with different roles and athorities on which methods to call and display appropriatte messages if errors occur.